Graduation Project Report

Byines — E-Commerce Platform for Luxury Modest Fashion

Design and Development of a Custom E-Commerce Platform for a Traditional Abaya Boutique

Academic Year: 2025 – 2026
Author: [Student Name]
Advisor: [Advisor Name]
Department: [Department Name]

Acknowledgments

[To be completed]

Table of Contents

General Introduction

E-commerce has transformed the retail landscape globally, and the modest fashion sector is no exception. Traditional abayas, a staple of elegant modest wear, represent a growing market that increasingly demands professional online shopping experiences. However, many boutique brands still rely on social media and messaging apps for sales, lacking structured catalogs, inventory management, and secure checkout.

This report documents the design and development of **Byines**, a custom-built e-commerce platform for a luxury modest fashion boutique. The platform replaces an informal sales process (Instagram, WhatsApp) with a complete digital storefront: structured product catalogs with multi-variant support, a session-based shopping cart, AJAX-driven checkout, customer dashboards, and a full administrative backend.

The report is organized into five chapters. Chapter 1 introduces the project context and objectives. Chapter 2 covers the system analysis, requirements, and design (UML, database schema, visual identity). Chapter 3 details the technical implementation of each module. Chapter 4 discusses challenges encountered and their solutions. Chapter 5 concludes with an evaluation and roadmap for future enhancements.

Chapter 1: Project Context

1.1 Project Origin

The boutique behind Byines specializes in high-end traditional abayas, kimonos, scarves, and niqabs under the tagline "*Timeless Elegance.*" Its product range includes casual and evening abayas crafted from premium materials such as Nida, Medina silk, and crepe.

Prior to this project, the boutique's digital presence was limited to Instagram and WhatsApp. Customers browsed photos manually, inquired about availability via direct messages, and placed orders through unstructured chat threads. This manual process caused:

- No centralized product catalog or inventory tracking
- Difficulty managing size/color variants and stock levels
- No formal order records or status tracking
- Limited ability to scale beyond local clientele

1.2 Objectives

Main Objective: Design and develop a custom, responsive, secure e-commerce platform that showcases the boutique's collections, delivers an intuitive shopping experience, and provides administrative tools for business operations.

Specific Objectives:

- Structured product catalog with categories, collections, and multi-variant support (color, size, stock)
- Session-based shopping cart with real-time AJAX updates
- Frictionless checkout with shipping calculation (Morocco-focused) and payment method selection
- WhatsApp order confirmation integration
- Customer dashboard with order history and profile management
- Full admin panel for products, categories, collections, orders, and sales overview

1.3 Scope

The platform covers:

- **Customer-facing**: Homepage, shop with filters/pagination, product detail with image gallery and variant selection, cart, checkout, order confirmation, user dashboard (order history, profile, account deletion)
- **Admin**: Tabbed single-page interface with overview stats, product CRUD with image gallery (drag-drop upload, color tagging, reorder), variant/stock management, category and collection CRUD, order management with status updates

---

Chapter 2: Analysis and Design

2.1 Needs Analysis

Analysis of the boutique's existing operations revealed several gaps:

- **No digital catalog**: Customers could not browse products by category or filter by attributes
- **No inventory visibility**: Stock levels were tracked manually, leading to overselling
- **No order tracking**: Once an order was placed via WhatsApp, there was no structured status workflow
- **No admin analytics**: The boutique lacked visibility into sales trends, revenue, or popular products

2.2 Problem Statement

> *How can we design and develop a lightweight, custom e-commerce platform that provides a premium shopping experience for luxury modest wear customers while equipping administrators with efficient management tools — without relying on heavy frameworks?*

Sub-questions:

1. How to structure a database to support multi-variant products (color/size combinations with individual stock)?
2. How to maintain visual excellence using vanilla CSS with a custom design system?
3. How to integrate local purchasing habits (Cash on Delivery, Moroccan shipping zones, WhatsApp) into a standard e-commerce flow?

2.3 Functional Requirements

Customer Features (Implemented)

- **Authentication**: Register, login, logout, password hashing
- **Product Catalog**: Category filtering, price sorting (low/high), pagination (12/page)
- **Product Detail**: Image gallery, color/size selection, dynamic price display, stock availability check
- **Shopping Cart**: Session-based, AJAX add/remove/update quantity, real-time badge count, subtotal calculation
- **Checkout**: Shipping form (Morocco), COD / PayPal selection, tax (10%), shipping cost (free Tangier, $3 elsewhere), AJAX order submission
- **Order Confirmation**: Display order summary, WhatsApp share button with preformatted message
- **User Dashboard**: Order history with status tracking, profile editing, account deletion
- **Reviews**: Read product reviews; submit rating and text
- **Wishlist**: Add/remove products via AJAX

Customer Features (Not Yet Implemented)

- Advanced search (name/description)
- Contact form / inquiries

- Address book management
- Password reset flow

Admin Features (Implemented)

- **Overview Dashboard**: Total orders, revenue, user count
- **Product Management**: Create, edit, delete products; upload images with drag-drop; assign images to colors; reorder images; manage variants (color, size, stock, price modifier); toggle visibility
- **Category Management**: Create, edit, delete with image upload
- **Collection Management**: Create, edit, delete with image upload; assign products by comma-separated IDs
- **Order Management**: View all orders, filter by status, view order details modal, update status, delete orders

Admin Features (Not Yet Implemented)

- Review moderation (approve/reject)
- User account management
- Sales charts / analytics (Chart.js not wired)
- Low-stock alerts

2.4 Non-Functional Requirements

- **Security**: All queries use PDO prepared statements; passwords hashed with `password_hash(PASSWORD_DEFAULT)` ; admin routes enforce `$_SESSION['user_role'] === 'admin'`
- **Responsiveness**: Fluid grids, mobile breakpoints (640px, 768px, 992px, 1024px)
- **Performance**: Static assets versioned with `?v=<?= time() ?>` ; CSS custom properties for theming
- **Browser Compatibility**: Chrome, Firefox, Safari, Edge
- **Code Organization**: OOP models in `classes/` , thin UI pages in `pages/` , CSS in separate stylesheets, JS in `scripts/`

2.5 Use Case Diagram

```
Actors: Customer, Administrator

Customer:
  - Browse Catalog
  - Filter by Category
  - View Product Detail
  - Select Color/Size
  - Add to Cart
  - Manage Cart (update qty, remove)
  - Checkout
  - View Order Confirmation
  - Share via WhatsApp
  - Register / Login / Logout
  - View Order History
  - Edit Profile
  - Delete Account
  - Add/Remove Wishlist
  - Submit Review

Administrator:
  - View Dashboard (orders, revenue, users)
  - Manage Products (CRUD, images, variants)
  - Manage Categories (CRUD)
```

```
  - Manage Collections (CRUD)
  - Manage Orders (view, filter, update status, delete)
```

2.6 Database Schema (ERD)

Tables (11 total):

Table	Purpose
`users`	Customers & admins (role: user/admin)
`categories`	Product categories (Abayas, Kimonos, Scarves, Niqab)
`products`	Core product data (name, slug, SKU, price, old_price)
`product_images`	Per-color images linked to products
`product_variants`	Color/size/stock/price_modifier combinations
`addresses`	User shipping addresses
`orders`	Order header (status, totals, shipping snapshot, payment)
`order_items`	Line items linked to variants
`wishlist`	User-product favourites
`reviews`	Ratings 1–5 with approval workflow
`collections`	Custom groupings (title, image, comma-separated product IDs)

Key Relationships:

- `categories` 1--N `products`
- `products` 1--N `product_images`, `product_variants`, `reviews`, `wishlist`
- `users` 1--N `orders`, `addresses`, `reviews`, `wishlist`
- `orders` 1--N `order_items`
- `product_variants` 1--N `order_items`

2.7 Visual Identity

Typography: `"Noto Serif", serif` for body and headings — conveys classic elegance. Material Symbols for icons.

Color Palette:

- `--brand-cream: #F4F1EE` — page background, soft luxury feel
- `--brand-earth: #D2C3B4` — borders, card accents
- `--brand-dark: #1A1A1A` — text, primary buttons, navigation
- `--gray-500: #6B7280` — secondary text, metadata
- `--white: #FFFFFF` — cards, modals, dropdowns

Design Principles: Minimalist, earth-toned, serif-driven; generous whitespace; subtle hover transitions and backdrop blur effects.

Chapter 3: Technical Implementation

3.1 Technology Stack

Layer	Technology
Backend	PHP 7.4+ (OOP models + procedural pages, no framework)
Database	MySQL / MariaDB via PDO
Frontend	Vanilla JavaScript (ES6+, no framework)
CSS	Custom CSS (CSS custom properties, Flexbox, Grid)
Fonts	Google Noto Serif, Material Symbols
Server	Apache (XAMPP)
Version Control	Git

3.2 Project Architecture

```
byines/
├── index.php                       # Redirects to pages/index.php
├── classes/                        # OOP domain models
│   ├── Database.php                # Singleton PDO wrapper
│   ├── User.php                    # Auth, profile, admin checks
│   ├── Product.php                 # CRUD, images, variants, reviews, related
│   ├── Category.php                # CRUD
│   ├── Cart.php                    # Session-based cart helper
│   ├── Order.php                   # Create, retrieve, update status
│   └── Collection.php              # Custom collection CRUD
├── includes/
│   ├── header.php                  # DOCTYPE, nav, autoloader, cart badge
│   └── footer.php                  # Footer, newsletter, copyright
├── pages/                          # 19 PHP pages
│   ├── index.php                   # Homepage (hero, popular picks, categories)
│   ├── shop.php                    # Collections with filters & pagination
│   ├── product.php                 # Product detail (gallery, variants, reviews)
│   ├── cart.php                    # Shopping cart
│   ├── cart_action.php             # AJAX cart handler
│   ├── checkout.php                # Checkout form
│   ├── process_order.php           # AJAX order submission
│   ├── order_success.php           # Order confirmation
│   ├── login.php / signup.php / logout.php
│   ├── dashboard.php               # User dashboard
│   ├── delete_account.php          # Account deletion handler
│   ├── admin_dashboard.php         # Single-page admin panel
│   ├── admin_manage_product.php    # Product CRUD handler
│   ├── admin_manage_category.php   # Category CRUD handler
│   ├── admin_manage_collection.php  # Collection CRUD handler
│   ├── admin_manage_order.php      # AJAX order status updates
│   └── admin_view_order.php        # Legacy order detail view
```

```
├── css/                            # 8 stylesheets
│   ├── style.css                   # Base, header, footer, homepage, shop
│   ├── product.css                 # Product detail page
│   ├── cart-checkout.css           # Cart & checkout
│   ├── dashboard.css               # User dashboard
│   ├── admin_dashboard.css         # Admin panel + order view
│   ├── login.css / signup.css      # Auth pages
│   └── cstyle.css                  # Legacy (partially used by shop page)
├── scripts/                        # JavaScript modules
│   ├── header.js, cart.js, checkout.js, product.js
│   ├── dashboard.js, admin_dashboard.js
│   └── toggle_password.js
└── products/                       # Per-product image directories
```

3.3 Database Layer

Database access uses the **Singleton pattern** (`Database::getInstance()`) to ensure a single PDO connection per request:

```php
class Database {
    private static $instance = null;
    private $conn;

    private function __construct() {
        $this->conn = new PDO("mysql:host=localhost;dbname=byines;charset=utf8mb4", "root",
"", [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false
        ]);
    }

    public static function getInstance() { ... }
    public function getConnection() { return $this->conn; }
}
```

Class-per-table mapping: Each domain class (`Product` , `Category` , `Order` , `User` , `Collection` , `Cart`) encapsulates all SQL queries for its table. Autoloading is handled via `spl_autoload_register` in `includes/header.php` .

3.4 Authentication & Session Management

Registration validates email uniqueness and encrypts passwords using `password_hash($pwd, PASSWORD_DEFAULT)` . Login uses `password_verify()` and stores `user_id` , `user_name` , and `user_role` in `$_SESSION` .

Admin routes check `$_SESSION['user_role'] === 'admin'` before rendering; non-admins are redirected.

The cart stores items in `$_SESSION['cart']` using composite keys: `$productId . '_' . md5($color . '_' . $size)` .

3.5 Product Catalog & Detail

Shop Page (`shop.php`):

- Fetches active products with optional category/collection filters
- Sorting: newest, price ascending, price descending
- Pagination: 12 products per page with prev/next navigation
- Responsive grid: 1 column (mobile) → 2 (tablet) → 4 (desktop)

Product Detail (`product.php`):

- Image gallery with main image + clickable thumbnails, color-tagged images
- Distinct color and size selectors derived from `product_variants`
- Dynamic price display: base price + optional variant modifier
- Old price / discount badge display
- Stock availability check per selected variant
- Related products section (same category, limit 4)
- Customer reviews section with star ratings

3.6 Shopping Cart

The `Cart` class (`classes/Cart.php`) manages session-based cart operations:

- Add item (with product_id, color, size, quantity, price)
- Update quantity
- Remove item
- Get subtotal, total count, all items
- Clear cart after order placement

All cart mutations are performed via AJAX (`cart_action.php`) returning JSON responses, enabling real-time badge count and summary updates without page reloads.

3.7 Checkout & Order Processing

Checkout Flow:

1. Customer fills shipping form (name, email, phone, address, city)
2. Selects payment method (Cash on Delivery or PayPal)
3. Order is submitted via AJAX to `process_order.php`

Order Processing (`process_order.php`):

- Reads cart data from session
- Calculates: subtotal, tax (10%), shipping ($0 Tangier, $3 elsewhere)
- Validates stock availability for each variant
- Uses **PDO transactions** to atomically: insert order → insert order items → decrement stock
- Generates order number format: `BYINES-XXXXXXXX` (hex string)
- Returns JSON with `order_id` on success

Order Confirmation (`order_success.php`):

- Displays order summary (number, date, items, totals, shipping details)
- WhatsApp share button linking to `wa.me` with preformatted order message

3.8 User Dashboard

Dashboard (`dashboard.php`):

- Tabbed interface: Overview, Order History, Account Details

- Overview: welcome banner, recent order widget with progress tracker, quick links
- Order History: full list with status badges, item details, order totals
- Account Details: edit profile form (first name, last name, email, phone), change password, delete account

3.9 Admin Panel

The admin panel (`admin_dashboard.php`) is a **single-page tabbed interface**:

- **Overview Tab**: Stats cards (total orders, revenue, user count)
- **Products Tab**: Table with ID, SKU, name, category, price, status (visible/hidden), Manage/Delete buttons
  - **Manage Product Modal** (3 tabs):
    - *Edit Details*: name, SKU, price, category, description, visibility
    - *Images*: Grid of uploaded images with star (set main), delete, reorder buttons; color assignment dropdown; drag-drop upload zone
    - *Stock*: Variant table (color, size, stock, price modifier); add new variant form; inline edit modal
- **Collections Tab**: Table + Add/Edit/Delete modals with image upload
- **Categories Tab**: Table + Add/Edit/Delete modals with image upload
- **Orders Tab**: Table with status badges, status filter dropdown, view detail modal with status update, delete

3.10 Frontend Styling Architecture

CSS is organized into **8 modular stylesheets** loaded per-page:

- `style.css` — Base reset, CSS variables, header, footer, homepage, shop, global utilities
- `product.css` — Product detail: gallery, variants, reviews, related products
- `cart-checkout.css` — Cart, checkout form, order summary
- `dashboard.css` — User dashboard panels
- `admin_dashboard.css` — Admin layout, tables, modals, forms, order management
- `login.css` / `signup.css` — Auth page styling

The design system uses **CSS custom properties** (`--brand-*` , `--gray-*` , `--font-*`) for consistent theming. All visual styling lives in stylesheets; inline styles are used only for dynamic PHP values (visibility toggles, progress widths, active borders).

Chapter 4: Challenges & Solutions

4.1 Database Transactions for Order Integrity

Problem: Order placement involves multiple dependent writes: insert order header, insert order items, decrement variant stock. Failure at any step would leave inconsistent data.

Solution: Wrapped the entire operation in a PDO transaction with commit/rollback:

```
$pdo->beginTransaction();
try {
    // Insert order
    $stmt = $pdo->prepare("INSERT INTO orders (...) VALUES (...)");
    $stmt->execute($params);
    $orderId = $pdo->lastInsertId();

    // Insert order items & decrement stock
    foreach ($cartItems as $item) {
```

```
        $stmtItem = $pdo->prepare("INSERT INTO order_items ...");
        $stmtItem->execute([$orderId, ...]);

        $stmtStock = $pdo->prepare("UPDATE product_variants SET stock_quantity =
stock_quantity - ? WHERE id = ?");
        $stmtStock->execute([$item['quantity'], $item['variant_id']]);
    }

    $pdo->commit();
} catch (Exception $e) {
    $pdo->rollBack();
    throw $e;
}
```

4.2 Stock Concurrency

Problem: Simultaneous orders for the same low-stock variant could result in negative inventory.

Solution: The variant's `stock_quantity` is checked inside the transaction before decrementing. If quantity exceeds available stock, the transaction rolls back and the customer receives an error.

4.3 AJAX Cart Management

Problem: Cart operations (add, update, remove, quantity change) needed to update the UI without page reloads.

Solution: A dedicated `cart_action.php` handler accepts POST requests, performs the operation via the `Cart` class, and returns JSON `{ success, newCount, subtotal, ...}`. JavaScript `fetch()` on the client updates the cart badge count, item list, and summary section.

4.4 Responsive Design

Problem: The luxury aesthetic requires careful adaptation to mobile screens without losing visual quality.

Solution: Used CSS Grid with responsive breakpoints:

- Product grid: 4 columns → 2 (tablet) → 1 (mobile)
- Cart layout: 2-column sidebar → single column stack
- Admin panel: 280px sidebar + content → full-width stacked
- Navigation: horizontal menu with dropdowns on desktop; collapsible on mobile

---

Chapter 5: Conclusion & Future Work

5.1 Project Evaluation

The Byines e-commerce platform successfully replaces the boutique's manual sales process with a complete digital solution. Customers can now browse a structured catalog by category, view detailed product pages with color/size options, manage a shopping cart, and complete checkout — all without leaving the website. Administrators have full control over products, inventory, categories, collections, and orders through a centralized panel.

The project demonstrates that a lightweight, framework-less stack (PHP + MySQL + vanilla JS) can deliver a polished, secure, and responsive e-commerce experience suitable for a boutique brand.

5.2 Features Implemented

Area	Feature	Status
Auth	Register, login, logout, session management	✅
Catalog	Category filtering, price sorting, pagination (12/page)	✅
Product	Image gallery, color/size selection, dynamic price, stock check	✅
Cart	Session-based, AJAX add/remove/update, badge count	✅
Checkout	Shipping form, COD/PayPal, tax/shipping calc, AJAX submit	✅
Confirmation	Order summary display, WhatsApp share button	✅
Dashboard	Order history, profile edit, password change, account deletion	✅
Reviews	Read & submit product reviews with star ratings	✅
Wishlist	AJAX add/remove	✅
Admin	Dashboard stats, product/category/collection CRUD, order mgmt	✅
Admin	Image gallery: drag-drop upload, color tagging, reorder	✅
Admin	Variant/stock management with inline editing	✅

5.3 Features Not Yet Implemented

Feature	Notes
Advanced search	Partial — name/description LIKE search not wired to UI
Contact form	No inquiries page or message handling
Address book	`addresses` table exists but no management UI in dashboard
Password reset	No "forgot password" flow
Review moderation	Admin cannot approve/reject reviews from panel
User management	Admin cannot view/manage customer accounts
Sales charts	Chart.js library available but not connected to real data
CSRF tokens	Not yet implemented (noted in RULES.md as required)
Email notifications	No transactional emails (order confirmation, shipping update)
Loyalty program	No rewards or points system
PWA	No service worker or manifest
Shipping logic unification	Two different implementations exist (Order.php vs process_order.php)

5.4 Future Enhancements

Priority	Enhancement

High	Add CSRF tokens to all state-changing forms
High	Unify shipping cost logic into a single source (`Order.php`)
Medium	Implement review moderation in admin panel
Medium	Add admin user management (view, disable accounts)
Medium	Wire Chart.js to real order data for sales analytics
Medium	Implement address book management in dashboard
Medium	Add `collections` table DDL to `setup-database.sql`
Medium	Unify order number format (BYINES-XXXX vs ORD-XXXX)
Low	Integrate email notifications via PHPMailer
Low	Integrate real payment gateway (Stripe, PayPal, CMI)
Low	Build Progressive Web App (PWA) for offline access
Low	Implement customer loyalty / rewards program
Low	Add search functionality with autocomplete

End of Report.